

FEMA Disaster Recovery Cost Forecasting Framework

End-to-End Machine Learning System for Public Sector Climate Risk Analytics

TerraNova Resilience Analytics Ltd | NEXYGENE

Project Lead: Stephen | July 2026

Python 3.13 • XGBoost • FastAPI • Streamlit • Kafka • Airflow • Docker • MLflow • NOAA

Executive Summary

The FEMA Disaster Recovery Cost Forecasting Framework is a production-grade machine learning system that predicts federal disaster recovery costs at the point of disaster declaration — before any funding commitments are made. Built for TerraNova Resilience Analytics Ltd under the NEXYGENE umbrella, it addresses a real gap in emergency management: the absence of a data-driven, declaration-time cost estimate to support early budget planning and resource allocation decisions.

The system integrates three external data sources — FEMA's Open API (declarations and public assistance funding), NOAA's Storm Events Database (wind speed and flood intensity), and NOAA's GHCN-Daily precipitation records (~78,500 US weather stations) — into a unified feature engineering pipeline, trains an XGBoost model tracked via MLflow, and serves predictions through a FastAPI backend and Streamlit dashboard, containerized with Docker.

A real-time streaming layer built on Apache Kafka provides an alternative production ingestion path alongside the batch pipeline, with Airflow DAGs for orchestration scheduling.

Headline result: XGBoost achieves a test R^2 of 0.513 on log-transformed recovery costs — an honest, audited figure that survived correction of a lookahead-bias defect, a severity-score target-leakage issue, and a training/serving scale mismatch, all discovered and fixed during development.

Contents

- 1. Project Overview & Motivation
- 2. System Architecture
- 3. Data Sources & Ingestion — Dual Architecture
- 4. Feature Engineering
- 5. Severity Score — Full Investigation & Redesign
- 6. Model Development & MLflow Tracking
- 7. Hyperparameter Tuning & Feature Additions
- 8. Lookahead Bias Fix — state_disaster_frequency
- 9. FastAPI Backend
- 10. Streamlit Dashboard
- 11. Kafka Streaming Layer
- 12. Airflow Orchestration
- 13. Containerization (Docker)
- 14. Validation & Sanity Checks
- 15. Key Findings & Honest Reporting
- 16. Known Limitations & Future Work
- 17. Technical Stack

1. Project Overview & Motivation

Disaster recovery in the United States is funded through FEMA's Public Assistance programme, which obligates federal dollars to states and localities following declared disasters. These obligations can range from hundreds of thousands of dollars for a minor severe storm to hundreds of millions for a major hurricane or compound flooding event. Emergency managers, state budget officers, and federal analysts must begin planning resource allocation immediately upon declaration — often days or weeks before any reliable cost estimate exists.

This project addresses that gap by training a predictive model on historical FEMA disaster declarations (comprehensive data from 1996) and enriching it with real climate intensity signals from NOAA, producing a single cost forecast at the moment a disaster is declared, based only on information genuinely available at that point in time.

Project Objectives

- Build a declaration-time cost forecasting model using only pre-declaration features
- Integrate multi-source climate data (storm events + precipitation) into a principled severity score
- Implement dual ingestion architecture: batch pipeline for training, Kafka streaming for production
- Track all experiments with MLflow for reproducibility and auditability
- Serve predictions through a production-ready FastAPI backend
- Visualize scenario analysis through an interactive Streamlit dashboard
- Containerize the full serving stack with Docker
- Identify and correct data quality issues (lookahead bias, target leakage, scale mismatches) with documented evidence

2. System Architecture

The system is structured in three phases: a dual ingestion layer (batch and streaming), a training pipeline (offline), and a serving stack (online, containerized).

Ingestion Layer — Dual Architecture

Path	Component	Mechanism	Output
Batch (training)	fetch_data_from_API.py	Paginated HTTP pull, incremental Parquet append	declarations/PA/summaries Parquet
Streaming (production)	kafka_producer.py	Same endpoints → Kafka topic (fema_raw)	Kafka messages
Streaming (production)	kafka_consumer.py	Consumes fema_raw, deduplicates, batches	features.parquet (gzip)

Training Pipeline

Component	Location	Purpose
NOAA Storm Events	src/ingestion/climate_ingest.py	Wind speed + flood events by FIPS + date
NOAA GHCN-Daily	src/ingestion/rainfall_data_ingestion.py	Daily precipitation by state + date
Feature Engineering	src/processing/clean_and_engineer.py	Cleans, merges, engineers all features including severity score
Model Development	ml/model_development.py	Trains LR/RF/XGBoost with RandomizedSearchCV + MLflow logging
Permutation Importance	ml/permutation_feature_importance_check.py	Bias-corrected importance via sklearn permutation method

Serving Stack (Docker)

Component	Container	Host Port	Purpose
FastAPI Backend	fema-cost-api	8001	Loads trained model, serves /predict-cost

Component	Container	Host Port	Purpose
Streamlit Dashboard	fema-cost-dashboard	8502	Scenario input UI, budget gap visualization
Inter-container	Docker network	8000 (internal)	Dashboard→API via service-name DNS

3. Data Sources & Ingestion — Dual Architecture

FEMA Open API — Three Endpoints

All three FEMA endpoints are consumed by both the batch and streaming ingestion paths, using the same pagination logic (\$skip/\$top, 1,000 records per page):

Endpoint	Records	Key Fields
DisasterDeclarationsSummaries	209,310 raw	disasterNumber, state, incidentType, dates, declarationType
PublicAssistanceFundedProjectsDetails	810,774 projects	disasterNumber, obligatedAmount, projectCategory
FemaWebDisasterSummaries	11,727 records	disasterNumber, totalFederalShareObligations (cross-validation)

Batch Path — `fetch_data_from_API.py`

The batch ingestion script streams each endpoint page-by-page and appends results incrementally to Parquet files. On each page, it reads the existing Parquet (if present), concatenates the new batch, and rewrites. This makes it safe to interrupt and resume — each run extends rather than overwrites the data.

Important limitation: the batch path has no deduplication. Running it twice will produce duplicate rows. It is designed for one-shot bulk acquisition of the full historical record, which is what the training pipeline requires. For repeated or ongoing ingestion, use the Kafka streaming path instead.

Streaming Path — Kafka Producer + Consumer

The Kafka pipeline is an architectural upgrade of the batch path for production real-time ingestion. The producer pulls the same three endpoints with the same pagination logic but routes records through a Kafka topic (`fema_raw`) rather than writing directly to disk. It adds schema versioning, UTC event timestamps, graceful SIGINT/SIGTERM shutdown, delivery confirmation via `future.get()`, and full environment variable configuration.

The consumer reads from `fema_raw`, deduplicates records by `disasterNumber` using an in-memory set loaded from the existing Parquet on startup, batches 500 records, and writes to Parquet with gzip compression. Both components run continuously and are designed for long-lived production operation.

Feature	Batch Path	Kafka Path
Same 3 FEMA endpoints	Yes	Yes
Incremental Parquet append	Yes	Yes
Deduplication	No	Yes (in-memory set)
Continuous operation	No (terminates when API exhausted)	Yes (runs indefinitely)
Graceful shutdown (SIGINT/SIGTERM)	No	Yes
Schema versioning on events	No	Yes (schema_version, event_time, payload)
Broker-mediated decoupling	No (direct HTTP→disk)	Yes (Kafka topic)
Environment variable config	No (hardcoded paths)	Yes (all configurable)
Delivery confirmation	No	Yes (future.get timeout=60s)

NOAA Storm Events Database

31 years of Storm Events detail files (1996–2026) downloaded from NOAA's public FTP server. Matched to FEMA disasters by county FIPS code + incident date-range overlap (± 3 -day buffer). Only county-level records (CZ_TYPE='C') used.

Metric	Value
Raw event records loaded	1,791,382
County-level records kept	1,059,514
Training disaster match rate	72.0% (1,272/1,766)
Signals produced	wind_speed (max in-county), flood_severity (event count)

NOAA GHCN-Daily (Rainfall)

31 years of GHCN-Daily by_year files (~150-185 MB each compressed), processed one year at a time to avoid memory exhaustion. Filtered to US-prefixed stations reporting PRCP, matched to FEMA disasters by state + incident date window.

Metric	Value
US precipitation stations	78,567 of 132,501 total
Total US PRCP rows processed	~187 million across 31 years
Training disaster match rate	95.8%
Signal produced	rainfall_intensity (peak daily mm in state)
Spatial precision	State-level (county-level deferred — requires geopandas)

Architecture note: The one-year-at-a-time processing pattern was required after the initial implementation crashed with an out-of-memory error (~65M accumulated rows). Processing load/match/aggregate/discard per year keeps peak memory constant regardless of span.

4. Feature Engineering

Feature engineering is performed by `clean_and_engineer.py`, producing a 62-column dataset from which 23 features are selected for modelling. All selected features represent information genuinely available at declaration time.

Feature Set

Category	Features	Notes
Temporal	declaration_year, declaration_month_sin/cos, declaration_quarter_sin/cos	Cyclical sin/cos encoding preserves month 12 ↔ month 1 adjacency
Duration & Lag	incident_duration_days, days_to_declaration	Top-2 features by permutation importance throughout all runs
Geographic	region_encoded	Census region label-encoded (Northeast/South/Midwest/West)
Severity	incident_severity_score	Climate-blended: 60% incident-type lookup + 40% NOAA climate intensity
Disaster History	state_disaster_frequency	Cumulative prior-disaster count per state (corrected for lookahead bias)
Incident Type Flags	is_hurricane, is_flood, is_fire, is_severe_storm, is_tornado, is_snowstorm, is_biological, is_severe_ice_storm, is_typhoon, is_drought	Top-10 incident types one-hot encoded
Interactions	severity_x_duration, severity_x_frequency, duration_x_days_to_declaration	Explicit interaction terms; prevent tree from needing many splits to find non-additive effects

Target Variable

Target: `log_total_obligated_amount = log1p(total_obligated_amount)`. Log-transformed because disaster costs are heavily right-skewed. Predictions are reversed with `expm1()` at serving time.

5. Severity Score — Full Investigation & Redesign

The severity score investigation is the most technically substantive part of the project. Three distinct problems were found and fixed in sequence.

Stage 1: Original Formula — Target Leakage

```
incident_severity_score = 0.4×incident_duration_days + 0.3×state_disaster_frequency
+ 0.3×total_obligated_amount.rank(pct=True)
```

The third component is a percentile-rank of total_obligated_amount — the target variable. This constitutes target leakage: the final obligated amount is only known months after declaration, not at prediction time.

Stage 2: Fixed Lookup Table — Scale Mismatch

Replaced leaking component with a fixed incident-type severity lookup (Hurricane=5 → Other=1). Eliminated leakage. But validation tests revealed the model was trained on raw integers 1–5 while the API sent floats 0–1. A payload with severity=0.95 returned the same estimate as severity=0.0, proving severity had no real influence when inputs were out-of-distribution.

Fix: Normalized the lookup to 0–1 using the dict's own min/max — not data-driven — producing {Other=0.0, Drought=0.25, Severe Storm=0.50, Flood=0.75, Hurricane=1.0}.

Stage 3: Redundancy — Type Flags Already Carry the Same Information

After fixing the scale mismatch, permutation importance still returned 0.0055 (statistical noise). The fixed lookup is a deterministic function of incidenttype — and the model already had the same information via 10 one-hot is_* flags. Severity was redundant. The fix required genuinely independent information: continuous NOAA climate intensity that differentiates disasters of the same type (e.g. category-1 vs. category-5 hurricane).

Stage 4: Climate-Blended Formula (Final)

```
incident_severity_score = 0.60 × type_severity_normalized
                        + 0.40 × mean(normalize(wind_speed),
                                      normalize(flood_severity),
                                      normalize(rainfall_intensity))
```

Evidence: Permutation Importance Trajectory

Stage	Permutation Importance	Rank
Original formula (leaked target)	Not measured (invalid)	—
Fixed lookup, raw 1–5 scale	0.0055 ± 0.0047	#9 of 18 (noise floor)
Fixed lookup, normalized 0–1	0.0055 ± 0.0047	#9 of 18 (redundancy unchanged)
Climate-blended (wind + flood)	0.1612 ± 0.0230	#4 of 18 (real signal)
Climate-blended (+rainfall, +interactions)	0.0712 ± 0.0151	#5 of 23 (credit split with interaction terms)

The jump from 0.0055 to 0.1612 represents a ~29× increase. When summing severity-related features after interaction terms were added: severity_x_duration (0.0916) + severity_x_frequency (0.0075) + incident_severity_score (0.0712) = 0.1703 combined — higher than 0.1612, confirming no real regression.

6. Model Development & MLflow Tracking

Three model types were trained and compared: Linear Regression (untuned baseline), Random Forest (RandomizedSearchCV, 50 iterations, 5-fold CV), and XGBoost (same). Fixed 80/20 split, random_state=42, 1,412 train / 354 test.

MLflow Setup

```
mlflow.set_tracking_uri('sqlite:///mlflow.db') # Windows-safe URI
mlflow.set_experiment('FEMA_Cost_Forecasting')
```

Each run logs: hyperparameters, R^2 /RMSE/MAE, best CV R^2 , model type tag, serialized model artifact. SQLite backend chosen over file-store after Windows path compatibility issues with MLflow's file:// URI handling.

Final Model Comparison (post-bias-fix, post-tuning, post-climate-enrichment)

Model	CV R^2	Test R^2	Test RMSE	Test MAE
Linear Regression	—	0.234	1.874	1.308
Random Forest	0.396	0.466	1.565	1.146
XGBoost (best)	0.386	0.513	1.494	1.092

Best XGBoost hyperparameters: subsample=0.6, reg_lambda=0.5, reg_alpha=0, n_estimators=500, min_child_weight=1, max_depth=6, learning_rate=0.01, gamma=0.3, colsample_bytree=1.0.

CV/Test Gap

A persistent ~0.10–0.13 R^2 gap between CV scores (~0.39–0.42) and test scores (~0.47–0.51) was observed across all runs. Consistent with a small-sample-size effect: 1,766 disasters, ~283 per CV fold. The test R^2 of 0.513 should be treated as a somewhat optimistic point estimate; repeated k-fold CV would produce a more stable, likely slightly lower, central estimate.

7. Hyperparameter Tuning & Feature Additions

Cyclical Date Encoding

declaration_month and declaration_quarter replaced with sin/cos transforms to preserve the cyclical adjacency of month 12 and month 1. After adding interactions, declaration_month_sin ranked #8 of 23 (importance 0.0519). Quarter sin/cos proved less useful and are candidates for pruning in future iterations.

Interaction Terms

Feature	Rank (final model)	Importance
severity_x_duration	#3	0.0916
duration_x_days_to_declaration	#6	0.0677
severity_x_frequency	#10	0.0075

Net Effect of Tuning

Phase	XGBoost Test R ²	Change
Pre-tuning (climate-enriched only)	0.591	—
Post-tuning + interaction features	0.598	+0.007
Post-lookahead-bias fix	0.513	−0.085 (bias removed, not performance regressed)

8. Lookahead Bias Fix — state_disaster_frequency

The original feature counted all disasters a state ever had across the full dataset — including disasters declared after the current row's declaration date. A Texas disaster from 2005 received the same frequency as a 2024 Texas disaster. This is lookahead bias.

The Fix

```
decl =  
decl.sort_values(['declarationdate', 'disasternumber']).reset_index(drop=True)  
decl['state_disaster_frequency'] = decl.groupby('state').cumcount()
```

cumcount() returns 0 for the first disaster in a state, 1 for the second, etc. — only counting disasters declared before the current one.

Measured Impact

Model	Test R ² (leaked)	Test R ² (corrected)	Drop
Linear Regression	0.312	0.234	−0.078
Random Forest	0.566	0.466	−0.100
XGBoost	0.598	0.513	−0.085

The consistent ~0.08–0.10 drop across all three structurally different model types is strong evidence of genuine future-information leakage. The corrected R² of 0.513 is the project's final, honest reported figure.

Permutation importance for state_disaster_frequency fell from 0.1388 (rank #5) to 0.0531 (rank #7) after the fix. Its interaction term severity_x_frequency fell correspondingly from 0.0365 to 0.0075.

9. FastAPI Backend

Endpoints

Endpoint	Method	Purpose
GET /health	GET	Returns {status:ok, model_loaded:true} — used by Docker HEALTHCHECK
POST /predict-cost	POST	Accepts 18 raw inputs, derives engineered features server-side, returns cost
GET /docs	GET	Auto-generated Swagger UI for interactive testing
GET /	GET	Root listing available routes

Server-Side Feature Derivation

The API derives all engineered features (cyclical encodings, interaction terms) inside the `predict_cost` handler from the 18 raw inputs the client sends. This prevents training/serving drift — callers never compute derived features themselves.

```
month = input_dict['declaration_month']
input_dict['declaration_month_sin'] = math.sin(2 * math.pi * month / 12)
input_dict['severity_x_duration'] = severity * duration
# ... all 5 derived features computed before model.predict()
```

Validated Severity Ladder (final model, duration=14, freq=10)

Incident Type	Severity	Predicted Cost
Other (baseline)	0.00	\$7,004,323
Drought	0.25	\$16,906,769
Severe Storm	0.50	\$25,981,839
Flood	0.75	\$46,137,326
Hurricane	1.00	\$60,392,192

10. Streamlit Dashboard

Interactive scenario planning interface at <http://localhost:8502>. Reads API_URL from environment variable, defaulting to <http://127.0.0.1:8000> for local development and resolved to <http://api:8000> inside Docker via service-name DNS.

Components

- Sidebar — incident type multiselect (supports compound disasters), FEMA region, duration, days-to-declaration, disaster frequency, severity slider with suggested baseline from fixed type map, declaration date picker
- Prediction panel — predicted cost, model type, timestamp
- Budget Gap Analysis chart — surplus/shortfall at 50/70/90/100/110/130% of forecast (planning aid, not a confidence interval)
- Scenario summary JSON — exact payload sent to API for reproducibility

The budget gap chart uses fixed percentage multipliers applied to the point prediction — not statistical confidence bounds. A genuine prediction interval (quantile regression or bootstrapped residuals) is identified as future work.

11. Kafka Streaming Layer

The Kafka pipeline provides a production-grade real-time ingestion path for the same three FEMA endpoints that the batch pipeline uses. It is an architectural upgrade — not a replacement — of `fetch_data_from_API.py`.

`kafka_producer.py`

- Pulls DisasterDeclarationsSummaries, PublicAssistanceFundedProjectsDetails, FemaWebDisasterSummaries
- Routes each record through the `fema_raw` Kafka topic with schema versioning (`schema_version`, `source`, `event_time`, `payload`)
- Synchronous delivery confirmation via `future.get(timeout=60)` — no silent data loss
- Retry strategy with exponential backoff on 429/5xx HTTP errors
- Graceful SIGINT/SIGTERM shutdown via `signal.signal()` handlers
- Full environment variable configuration (`KAFKA_BOOTSTRAP`, `KAFKA_TOPIC`, `PAGE_SIZE`, etc.)

`kafka_consumer.py`

- Continuously reads from `fema_raw` topic, `group_id=fema-stream`
- Deduplicates by `disasterNumber` using an in-memory set loaded from existing Parquet on startup
- Batches 500 records, writes to `features.parquet` with gzip compression
- `try/finally` block ensures remaining batch saves on any exit condition
- Known limitation: the `processed_ids` set grows unboundedly — a Redis SET or Bloom filter would be needed for very long-running production deployments

Comparison: Batch vs. Streaming Path

Feature	<code>fetch_data_from_API.py</code> (batch)	Kafka pipeline (streaming)
Same 3 FEMA endpoints	Yes	Yes
Incremental Parquet append	Yes	Yes
Deduplication	No	Yes
Continuous operation	No	Yes
Graceful shutdown	No	Yes
Schema versioning	No	Yes

Feature	fetch_data_from_API.py (batch)	Kafka pipeline (streaming)
Broker decoupling	No	Yes
Env-var config	No	Yes
Used by training pipeline	Yes	No (separate output path)

12. Airflow Orchestration

An Airflow DAG (`airflow/dags/fema_pipelin.py`) provides scheduling and orchestration for the training pipeline, enabling automated periodic retraining as new FEMA data becomes available. The DAG encapsulates the full pipeline sequence: FEMA ingestion → NOAA climate ingestion → feature engineering → model training.

Note: the DAG filename contains a typo (`fema_pipelin.py` rather than `fema_pipeline.py`) — flagged for correction in a future commit.

13. Containerization (Docker)

Two separate images — API and dashboard — orchestrated by docker-compose. The training pipeline, NOAA data (~5GB), and mlflow.db are excluded from both images via .dockerignore.

Image Design

- Base image: python:3.11-slim — minimal footprint
- Dependencies installed before code copy — Docker layer caching means pip install only re-runs when requirements files change
- Only api/ and models/ copied into API image — ~5GB of cached NOAA data excluded
- HEALTHCHECK on both images using /health and /_stcore/health endpoints respectively
- XGBoost pinned to 3.2.0 — pickle files are not cross-version compatible

Port Configuration

Service	Internal Port	Host Port	Note
API	8000	8001	8000 occupied on development host
Dashboard	8501	8502	8501 occupied on development host
Inter-container	8000	N/A	Service-name DNS; unaffected by host mapping

Windows BuildKit Workaround

Docker Desktop's BuildKit/Bake API on Windows produces a 'transferring dockerfile: 2B' error due to a WSL2 path-resolution issue. Workaround: set DOCKER_BUILDKIT=0 to use the legacy classic builder.

```
$env:DOCKER_BUILDKIT=0
docker-compose up --build
```

14. Validation & Sanity Checks

Five boundary-case payloads were run end-to-end through the API to validate model behavior before deployment. The severity ladder was re-validated on the final model after each major change.

Five-Point Boundary Validation (early model)

Scenario	Severity	Duration	Predicted Cost
Minimal severe storm	0.02	1 day	\$460,619
Slow-burn drought	0.15	120 days	\$6,615,760
Small fire	0.32	10 days	\$28,125,815
Flood + hurricane	0.62	14 days	\$60,652,217
Extreme compound (4 flags)	0.98	90 days	\$81,428,424

Key Validation Finding — Severity Isolation Test

A targeted test held all is_* flags at zero and flipped only incident_severity_score from 0.0 to 1.0. Result: \$48.6M → \$150.3M — a 3.1× increase driven purely by severity with no type-flag confound. This confirmed climate blending produced real, independent predictive signal.

Containerized Stack Validation

Final validation ran the hurricane+flood payload through the full Docker stack (dashboard → API container → model → response) and confirmed \$50,802,971 — consistent with local predictions for equivalent inputs, confirming the Docker network and inter-container API calls were correctly configured.

15. Key Findings & Honest Reporting

Finding 1: Three Data Quality Issues Diagnosed and Corrected

- Target leakage in severity score (`total_obligated_amount.rank()` as a component) — eliminated
- Training/serving scale mismatch (1–5 integers vs. 0–1 floats) — corrected by normalizing the lookup
- Lookahead bias in `state_disaster_frequency` (full-dataset count) — corrected to cumulative count

Each issue was diagnosed with explicit, reproducible evidence — targeted API payloads, isolation tests, before/after permutation importance comparisons — not inferred from code review alone.

Finding 2: Climate Integration Resolved Severity Score Redundancy

Before NOAA integration: 0.0055 permutation importance (noise). After blending wind speed + flood events: 0.1612 (rank #4, ~29× increase). Confirmed that the feature required independent, continuous climate information to be non-redundant with the existing incident-type one-hot flags.

Finding 3: MDI Systematically Understated Low-Cardinality Features

The default `feature_importances_` (Mean Decrease Impurity) reported `incident_severity_score` at 2.34% even after the climate fix. Permutation importance correctly reported 16.12%. Reason: MDI is biased toward high-cardinality features. `incident_severity_score` has 5 unique values; `days_to_declaration` has 153. Permutation importance measures actual R^2 degradation on held-out data and is not susceptible to this bias.

Finding 4: The Bias-Corrected R^2 is the Only Defensible Figure

The final honest test R^2 is 0.513. The 0.591–0.598 figures reported earlier are attributable to approximately 0.08 R^2 of lookahead-bias leakage in `state_disaster_frequency`. Both numbers are reported transparently — the progression itself is a finding.

Finding 5: The Dual Ingestion Architecture Has Clear, Non-Overlapping Roles

`fetch_data_from_API.py` (batch) and the Kafka pipeline (streaming) pull the same data but serve different operational needs. The batch path is optimized for one-shot training data acquisition; the Kafka path is optimized for production real-time ingestion with delivery guarantees and deduplication. They are complementary, not redundant.

16. Known Limitations & Future Work

Limitation	Impact	Status
n=1,766 training disasters	CV/test gap (~0.10); test R^2 may be somewhat optimistic	Documented; future work: broader date range
Batch path lacks deduplication	Running <code>fetch_data_from_API.py</code> twice produces duplicates	Documented; use Kafka path for repeated ingestion
Rainfall join is state-level	Lower spatial precision than Storm Events county FIPS join	Documented; future work: geopandas county join
NOAA data starts 1996	~18% of dataset has no climate match	Documented; <code>fillna(0)</code> applied as neutral default
Budget gap chart is not a confidence interval	Fixed % bands are not probabilistic bounds	Future work: quantile regression or bootstrap intervals
<code>declaration_year</code> at rank #2	May encode inflation/programme growth, not risk	Future work: inflation-adjust target to 2026 dollars
No runtime severity validation	API accepts <code>severity=0.0</code> with <code>is_hurricane=1</code>	Future work: Pydantic validator to derive severity from flags
<code>kafka_consumer</code> memory growth	<code>processed_ids</code> set grows unboundedly	Future work: Redis SET or Bloom filter
Airflow DAG filename typo	<code>fema_pipelin.py</code> should be <code>fema_pipeline.py</code>	Flagged for correction

Priority Future Work

- Prediction intervals — quantile regression or bootstrapped residuals to replace arbitrary budget-gap bands
- County-level rainfall join — geopandas + Census TIGER/Line shapefiles
- Inflation adjustment — deflate `total_obligated_amount` to constant 2026 dollars
- Server-side severity derivation — derive `incident_severity_score` from type flags in the API, eliminating inconsistent input possibility
- Repeated k-fold CV — more stable R^2 estimate than single 20% split

17. Technical Stack

Category	Technology	Version	Purpose
Language	Python	3.13	All pipeline, API, and dashboard code
OS / Shell	Windows 11 / PowerShell	—	Development environment
ML Framework	XGBoost	3.2.0	Best-performing model
ML Framework	scikit-learn	1.5.2	RF, LR, preprocessing, permutation importance
Experiment Tracking	MLflow	—	SQLite backend; all runs tracked
API Framework	FastAPI	0.115.0	Prediction endpoint + auto Swagger docs
API Server	Uvicorn	0.32.0	ASGI server for FastAPI
Dashboard	Streamlit	1.39.0	Interactive scenario planning UI
Visualization	Plotly	5.24.1	Budget gap bar chart
Data Processing	Pandas	2.2.3	All dataframe operations
Data Processing	NumPy	1.26.4	Numerical operations, log transforms
Storage	Parquet (PyArrow)	—	Raw and processed data storage
HTTP Client	Requests	2.32.3	FEMA/NOAA API ingestion
Streaming	Apache Kafka (kafka-python)	—	Real-time FEMA data ingestion
Orchestration	Apache Airflow	—	Pipeline scheduling via DAGs
Containerization	Docker Desktop + Compose	—	Two-service container stack
Model Persistence	Joblib	1.4.2	Model and feature column serialization
Data Source	FEMA Open API	v1/v2	Declarations, PA funding, summaries
Data Source	NOAA Storm Events	NCDC	Wind speed, flood event data
Data Source	NOAA GHCN-Daily	NCEI	Daily precipitation, 78K+ US stations

FEMA Disaster Recovery Cost Forecasting Framework

TerraNova Resilience Analytics Ltd | NEXYGENE | July 2026

Confidential — Portfolio Documentation